

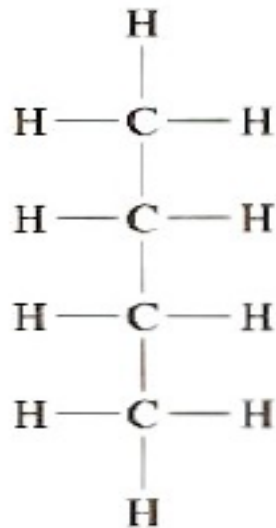
Trees

Introduction to Trees

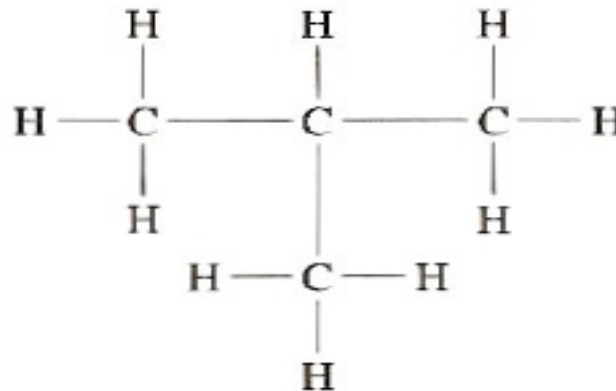
- ต้นไม้ (*tree*) สามารถใช้จำลองโครงสร้างต่างๆได้ เช่น โครงสร้างของสารประกอบอิมัลล์ โครงสร้างขององค์กร ระบบแฟ้มในคอมพิวเตอร์ เป็นต้น
- ต้นไม้(*tree*) คือกราฟเชื่อมต่อแบบไม่มีทิศทาง(*connected undirected graph*) ที่ไม่มีวงจร
 - **Theorem:** มีทางเดินอย่างง่ายเพียงเส้นทางเดียวระหว่างสองโหนด(จุด)ใดๆ
- ป่า(*forest*)คือเซตของต้นไม้ที่ไม่มีโหนด(จุด)ร่วมกัน
- จุดใบไม้(*leaf*) ในต้นไม้หรือป่า เป็นจุดที่เป็น pendant หรือจุดสันโดด (*isolated*) จุดภายใน(*internal*)คือจุดใดๆที่ไม่ใช่จุดใบ (ดังนั้นจุดภายในมีดีกรี ≥ 2)

Basic Definition

- ในวิชาเคมี ใช้กราฟแทนโมเลกุล
 - จุดยอดของกราฟแทนอะตอม
 - แต่ละด้านแทนพันธะ(bond) ระหว่างอะตอม



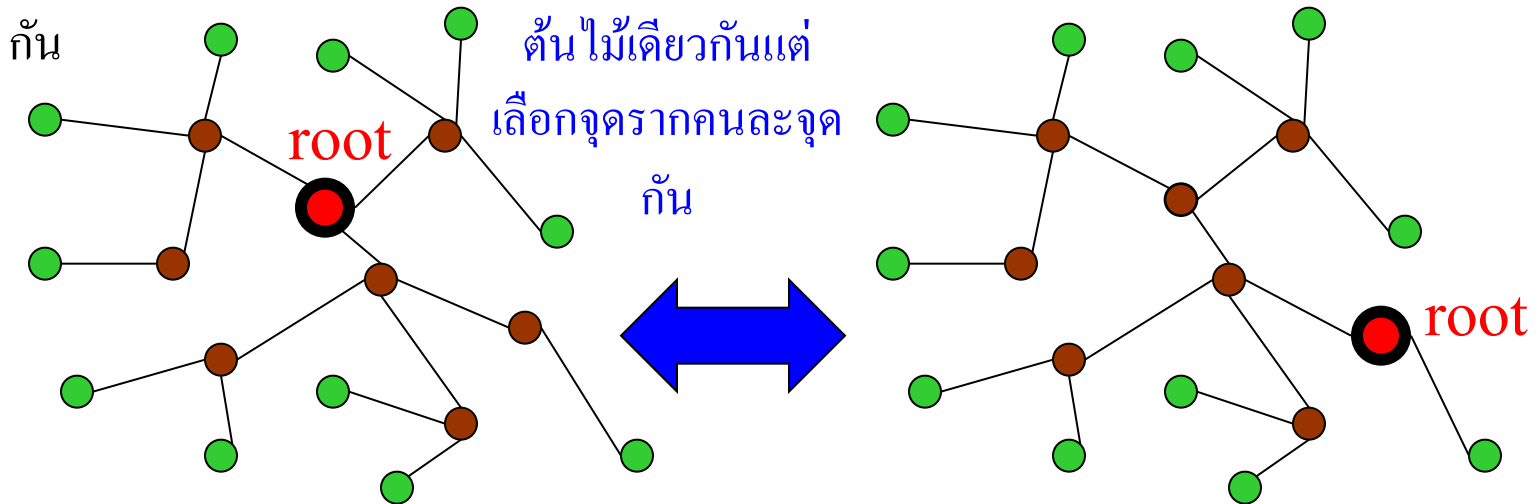
Butane



Isobutane

Rooted Trees

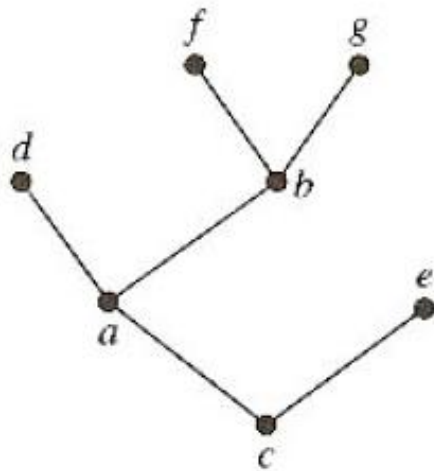
- ต้นไม้ที่มีราก(*rooted tree*) คือ ต้นไม้ที่มีการระบุให้จุดใดจุดหนึ่งเป็นราก (*root*) โดยทุกๆ ด้านจะมีทิศทางออกจากจุดราก เมื่อกำหนดให้จุดๆ หนึ่งเป็นรากของต้นไม้แล้ว เราสามารถจะกำหนดทางเดินที่เฉพาะเจาะจง(*unique*) จากรากไปยังจุดยอดอื่น ๆ ในต้นไม้ได้เสมอ
- สังเกตว่าต้นไม้ที่ไม่มีรากที่มี n โหนด จะสามารถสร้างเป็นต้นไม้ที่มีรากได้ n ต้นที่แตกต่างกัน



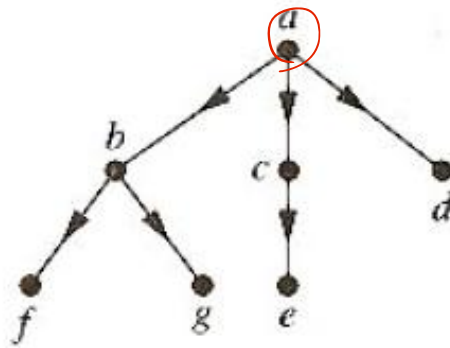
จุดใบแทนด้วยจุดสีเขียว จุดภายในแทนด้วยจุดสีน้ำตาล

Rooted Trees

- Example



T



With root a



With root c

คำศัพท์ที่เกี่ยวข้องกับต้นไม้

พ่อ/แม่ (Parent)

ถ้า T เป็นต้นไม้แบบมีราก และ V เป็นบัพใน T ที่ไม่ใช่ราก แล้ว พ่อ/แม่ของบัพ V คือ บัพ u เมื่อมีทางเดินจากบัพ u ลงมาหาบัพ V โดย u และ V ประชิดกัน และ u อยู่ใกล้รากมากกว่า V

ลูก (Child)

เมื่อ u เป็นพ่อ/แม่ของ V แล้วเราเรียก V ว่าลูกของ u

พี่น้องกัน (Siblings)

จุด 2 จุดหรือมากกว่า 2 จุด ที่มีพ่อ/แม่เดียวกันเรียกว่าพี่น้อง

คำศัพท์ที่เกี่ยวข้องกับต้นไม้

บรรพบุรุษ (Ancestor)

บรรพบุรุษของจุดใด ๆ ที่มี ไม่ใช่รากของต้นไม้ คือจุดทุกจุดที่อยู่บนวิถีจากรากจนถึงจุดนั้น

ลูกหลาน (Descendant)

ลูกหลานของจุด V คือ จุดทุก ๆ จุดที่มี V เป็นบรรพบุรุษ

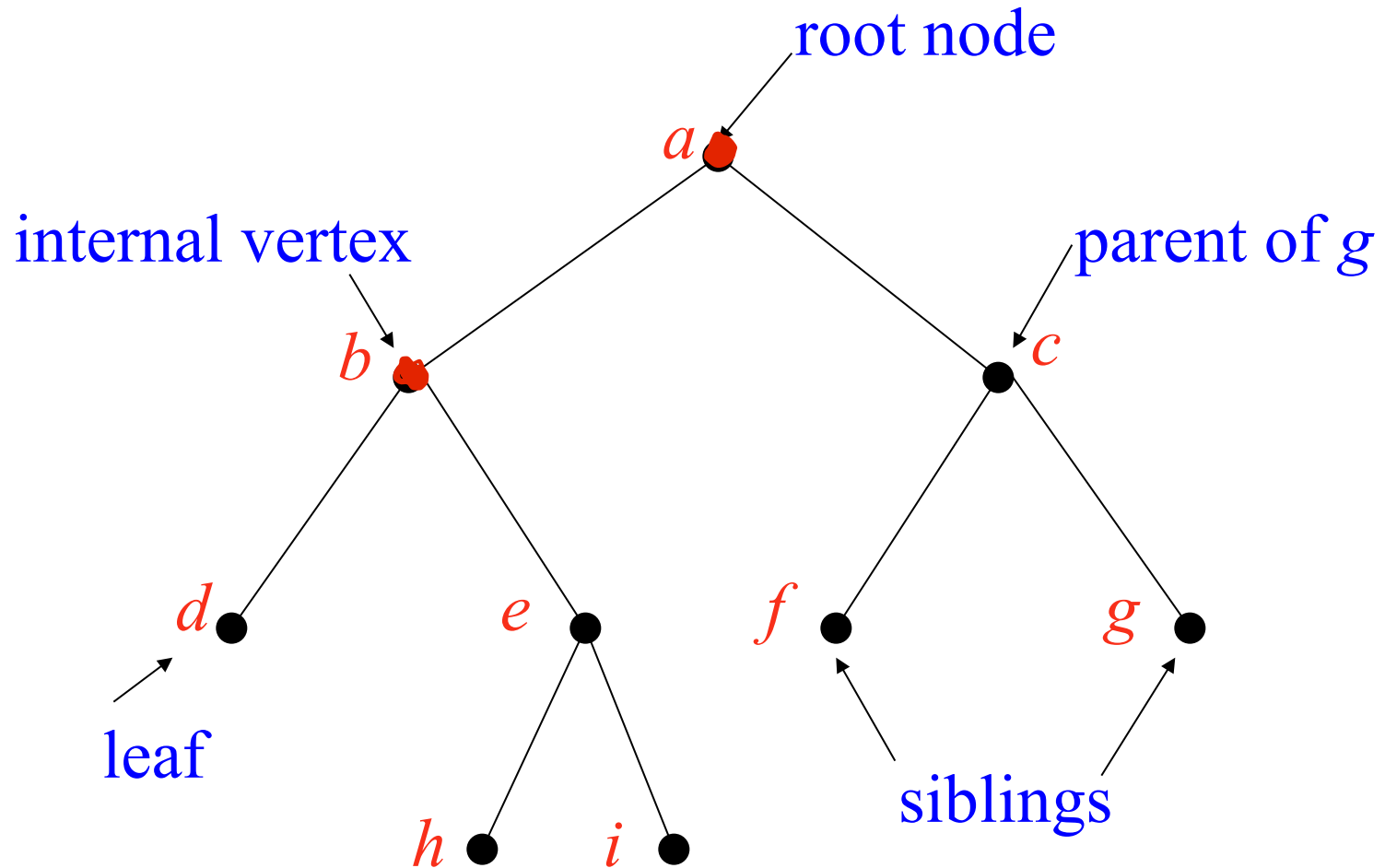
ใบ (Leaf)

จุดใด ๆ ที่ไม่มีลูก เรียกว่าใบ

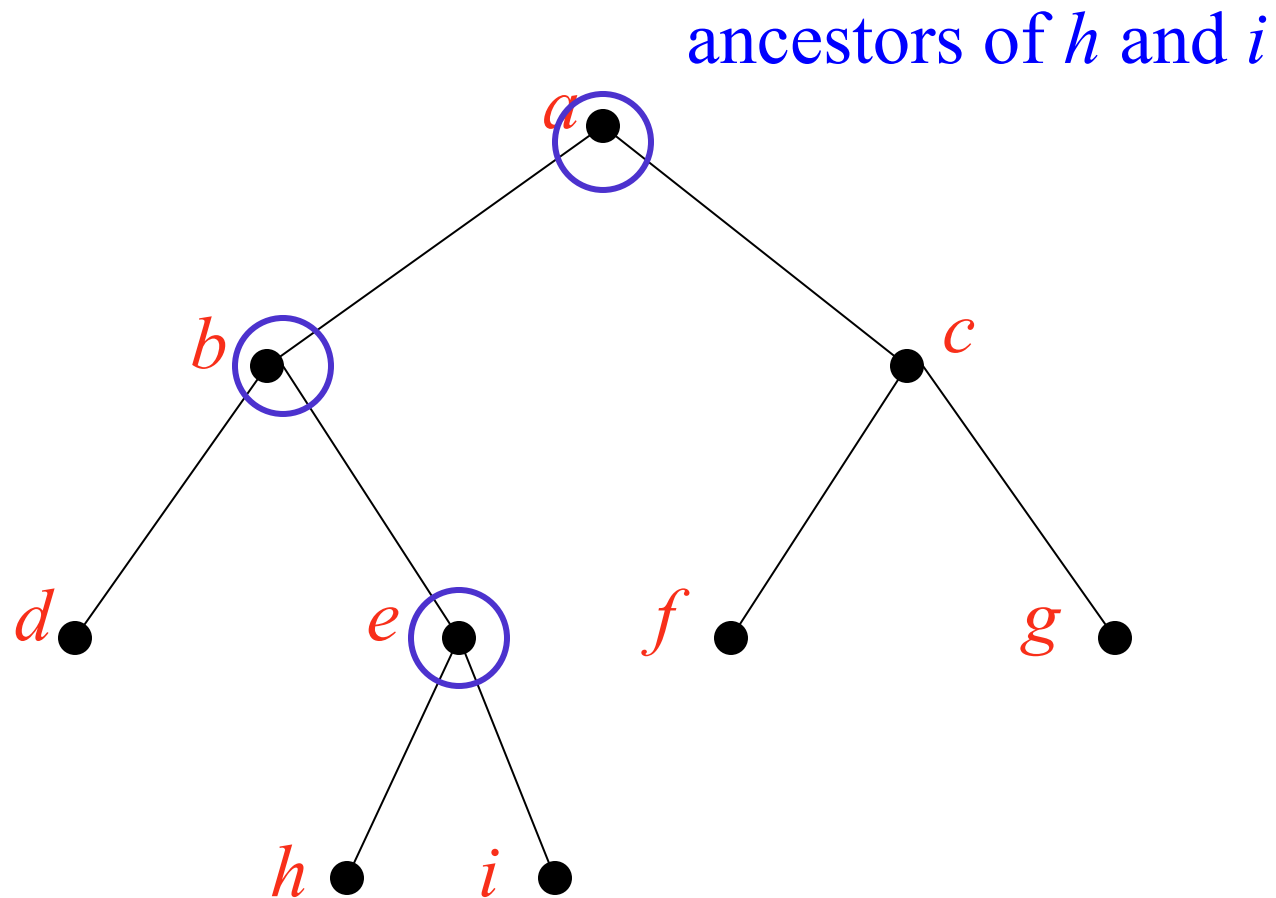
จุดภายใน (Internal vertex/nodes)

จุดทุกจุดที่มีลูก คือจุดภายในหรือเราอาจกล่าวได้ว่า จุดภายในคือจุดที่ไม่เป็นใบ

Rooted-Tree Terminology Example

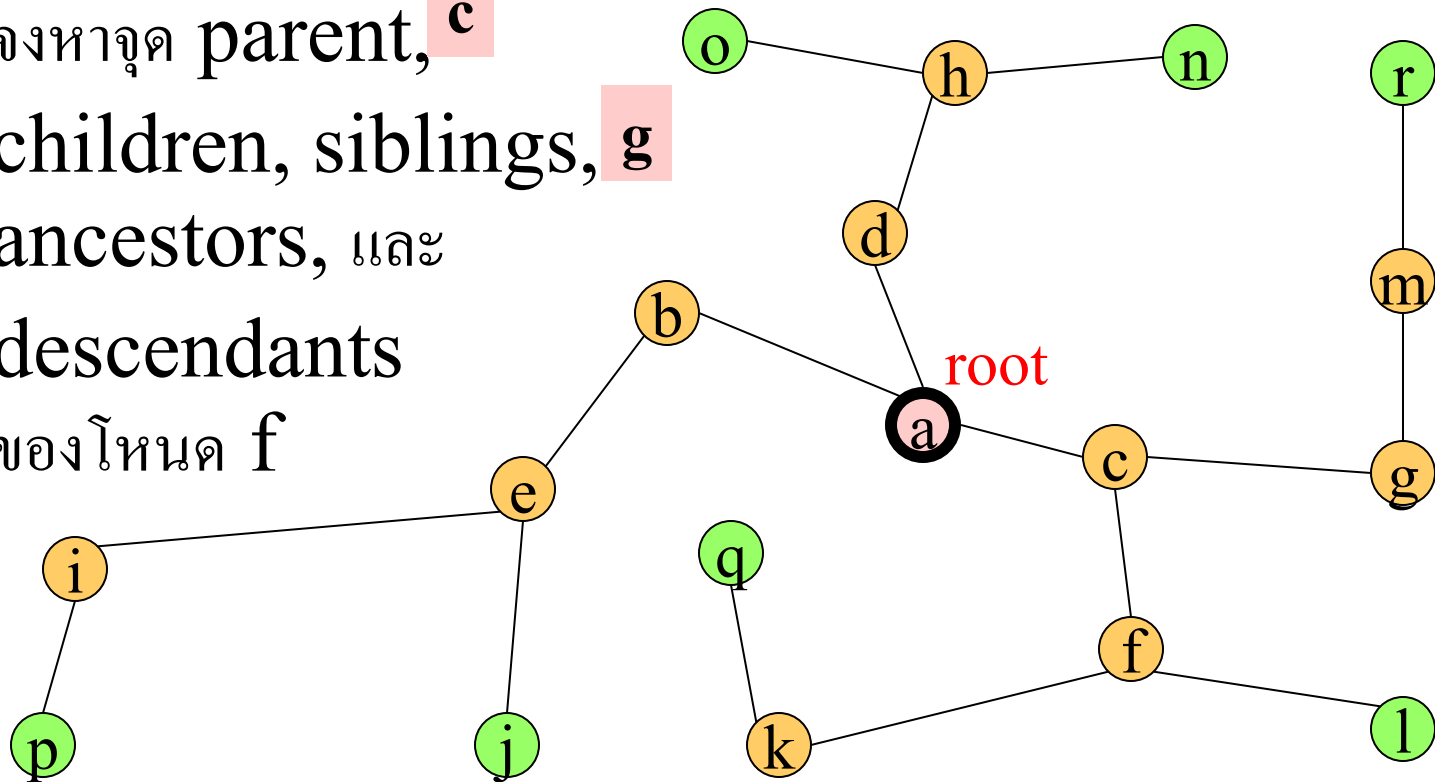


Rooted-Tree Terminology Example



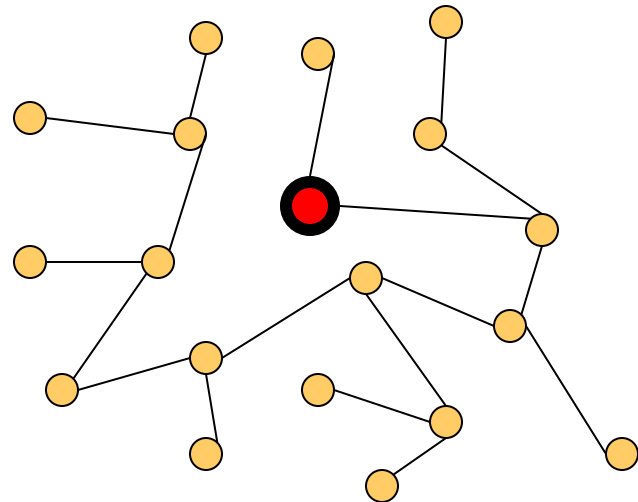
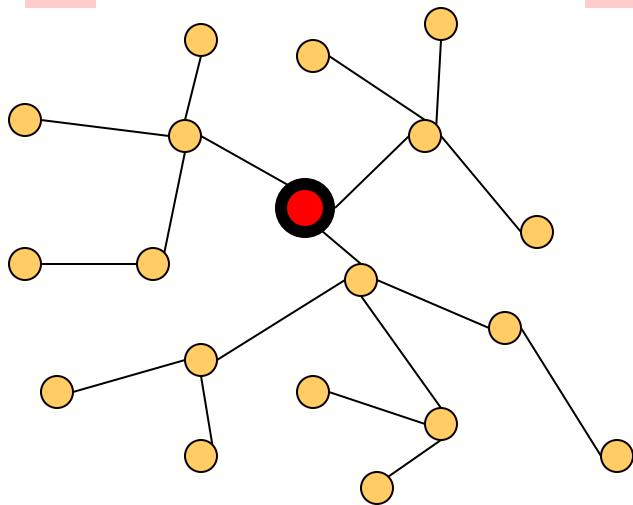
Rooted-Tree Terminology Exercise

- จงหาจุด parent, **c**
k, l children, siblings, **g**
a, c ancestors, และ
l, k, q descendants
ของโหนด f

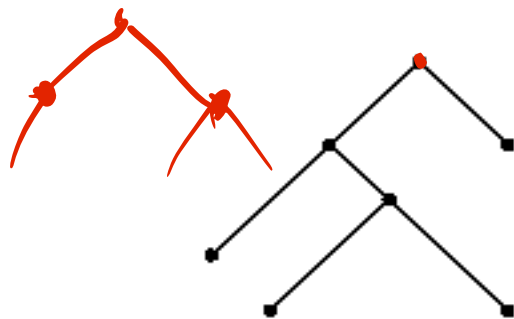


n -ary trees

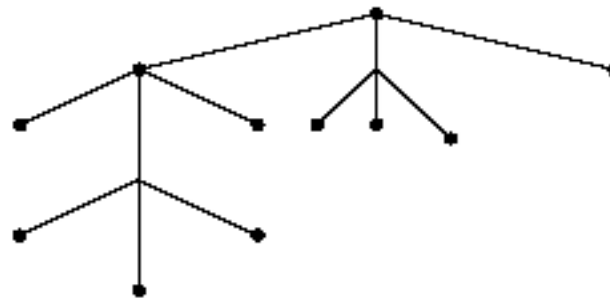
- ต้นไม้ที่มีราก จะเรียกว่า n ภาค(n -ary tree) ถ้าทุกๆจุดมีลูก(children)ไม่มากกว่า n จุด และ เรียกว่าต้นไม้ n ภาคเต็มต้น(*full n -ary tree*) ถ้าทุกๆจุดภายในมีลูกเท่ากับ n จุด
- ต้นไม้ 2-ary(มีลูกไม่เกิน 2 จุด) จะเรียกว่าต้นไม้ทวิภาค(*binary tree*)
- ทฤษฎี: ต้นไม้ที่มีรากเป็น binary tree ก็ต่อเมื่อทุกๆจุด(ยกเว้นจุดราก) มีดีกรี $\leq$ 3 , และจุดรากมีดีกรี $\leq$ 2 .



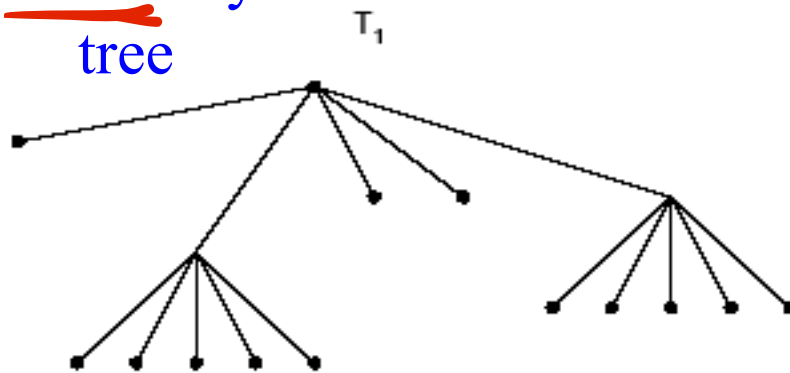
ต้นไม้เป็นต้นไม้แบบ m ภาคเต็มต้นไม้?



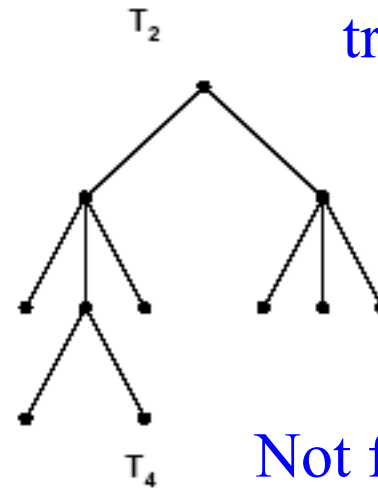
Full 2-ary
tree



Full 3-ary
tree



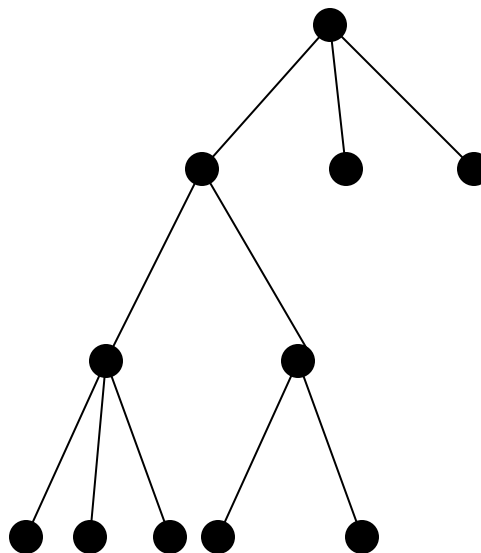
Full 5-ary
tree



Not full 3-ary
tree

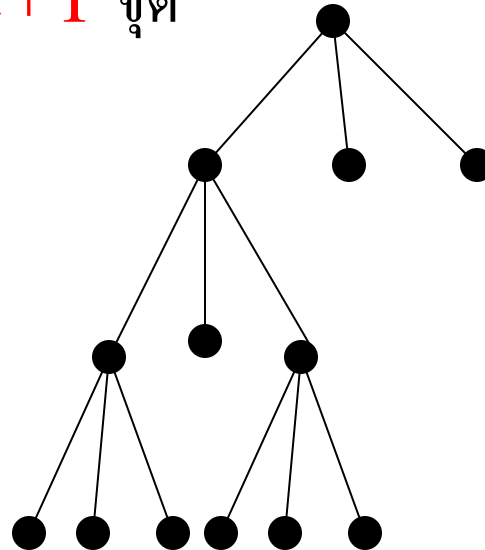
Some Tree Theorems

- ทฤษฎีบท 1 ต้นไม้ที่มีทั้งหมด n จุด จะมีจำนวนกิ่ง(ด้าน)
- $e = n - 1$ ด้าน



Some Tree Theorems

- ทฤษฎีบท 2 ต้นไม้ m ภาคเต็มต้น (full m -ary tree) ที่มีจุดภายในจำนวน i จุด จะมีจำนวนจุดทั้งหมด $n = mi + 1$ จุด และมีจุดใบ $l = (m - 1)i + 1$ จุด



Some Tree Theorems

- ทฤษฎีบท 3 ต้นไม้แบบ m ภาคที่มีจุดทั้งหมด n จุดและมีจุดภายในจำนวน i จุดและมีจุดใบจำนวน $\square$ จุด จะได้ว่า
- $i = (n-1)/m$ และ $1 = [(m-1)n+1]/m$
- $n = mi + 1$ จุด และ $1 = (m-1)i + 1$
- $n = (m\square - 1)/(m-1)$ และ $i = (1-1)/(m-1)$

$$n = m_i + 1$$

$$n = (n - l - 1)i + 1$$

$$l = n - m - 1$$

$$n = (n - 1 - 1)4 + 1$$

$$n = \text{ตัวอย่าง}$$

$$4n - 4 - 4 + 1$$

$$m = n - l - 1$$

- นางสาวตะวันฉายเป็นผู้เริ่มส่งจดหมายลูกโซ่ช่วยพรวันเกิดออกไปยังเพื่อน ๆ และขอร้องว่าให้แต่ละคนที่ได้รับจดหมายส่งจดหมายต่อไปถึงคนอื่น ๆ อีก ~~4~~ ⁴ คน อยากทราบว่าจะมีคนทั้งหมดกี่คนที่ได้อ่านจดหมายฉบับนี้รวมทั้งนางสาวตะวันฉาย ถ้ากำหนดว่าไม่มีใครได้รับจดหมายเกินหนึ่งฉบับ และเมื่อกระบวนการส่งจดหมายลูกโซ่สิ้นสุดลง มีทั้งหมด 100 คนที่ได้อ่านจดหมายนี้ แต่ไม่ยอมส่งจดหมายต่อ

$$n = +n = -n$$

$$-n$$

ตัวอย่าง

กระบวนการส่งจดหมายลูกโซ่เปรียบได้กับต้นไม้ 4 ภาค และจุดภายในคือ คนที่ส่งจดหมายต่อในขณะที่ไปคือผู้ที่ได้รับจดหมายแล้วไม่ส่งต่อ ให้ i แทนจำนวนจุดภายใน และ n แทนจำนวนคนทั้งหมด ในที่นี้ $m = 4$ จากทฤษฎี จะได้ว่า

$$\begin{aligned} n &= (mk-1)/(m-1) \\ &= [(4 \times 100)-1]/(4-1) = 133 \end{aligned}$$

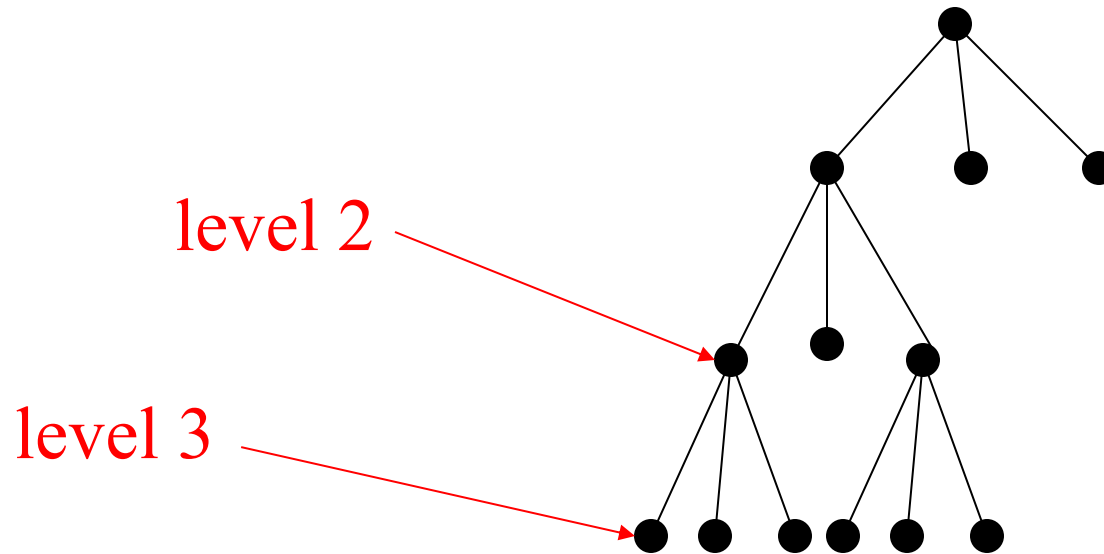
ดังนั้นมีผู้ได้รับจดหมายทั้งหมด 133 คน

$$\begin{aligned} \text{และ } i &= (k-1)/(m-1) \\ &= (100-1)/(4-1) = 33 \end{aligned}$$

ดังนั้นจำนวนคนที่ส่งจดหมายต่อมีทั้งหมด 33 คน

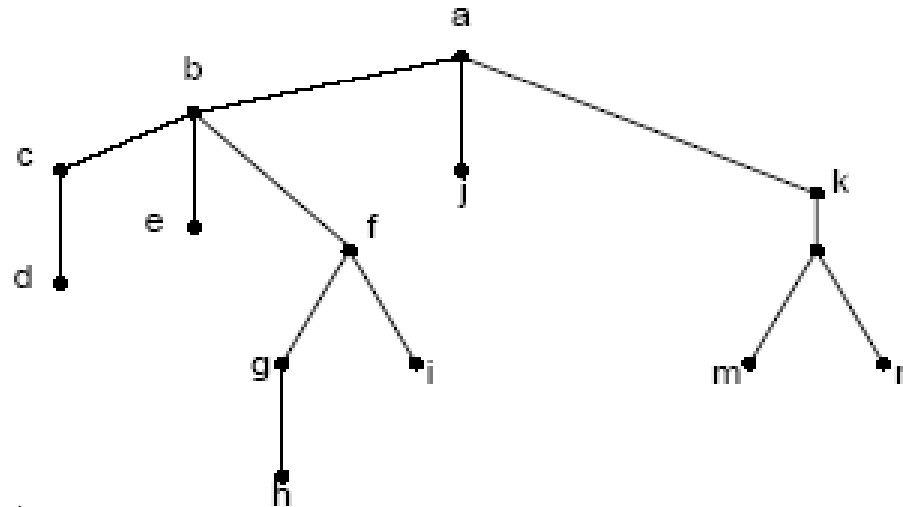
Some More Tree Theorems

- นิยาม: ระดับ(*level*) ของจุดคือความยาวของทางเดินอย่างง่ายจากราก (*root*) ไปยังจุดนั้น



Some More Tree Theorems

- ความสูง(*height*) ของต้นไม้ คือระดับของจุดที่มากที่สุด



ระดับของราก (จุด a) = 0

ระดับของบัพ b, j, k = 1

ระดับของบัพ d, g, i, m, n = 3

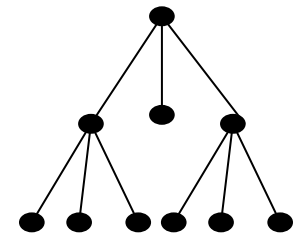
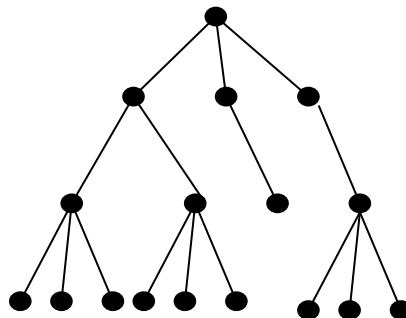
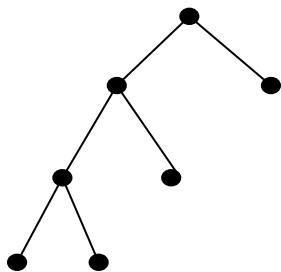
ระดับของบัพ c, e, f, l = 2

ระดับของบัพ h = 4

ดังนั้น ความสูงของต้นไม้ $T = 4$

Some More Tree Theorems

- ต้นไม้ m ภาค(m -ary tree) ที่มีราก ที่มีความสูง h เรียกว่าต้นไม้สมดุล (*balanced*) ถ้าจุดใบทุกจุดอยู่ที่ระดับ h หรือ $h-1$
- ทฤษฎีบท: มีจำนวนใบอย่างมากที่สุด m^h ใบ ในต้นไม้ m ภาค(m -ary tree) ที่มีความสูง h
 - ทฤษฎีบทย่อย : ต้นไม้ m -ary ที่มีจุดใบ 1 ใบมีความสูง $h \geq \lceil \log_m l \rceil$ ถ้าเป็นต้นไม้เต็มกิ่ง(full m -ary) และสมดุลความสูง $h = \lceil \log_m l \rceil$

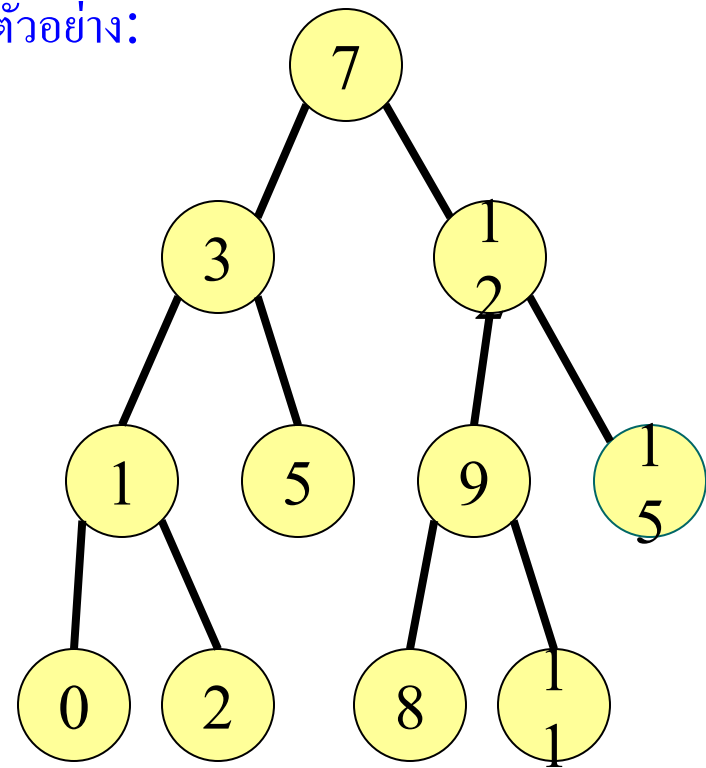


Applications of Trees

Binary Search Tree Format

- ข้อมูลแต่ละตัวจะถูกเก็บลงในจุดของต้นไม้ โดยการจัดเรียงข้อมูลในต้นไม้สำหรับทุกๆค่า x คือ,
 - ทุกๆจุดที่อยู่ในต้นไม้ย่อยด้านซ้าย (left subtree) ของ x จะมีค่าน้อยกว่า x
 - ทุกๆจุดที่อยู่ในต้นไม้ย่อยด้านขวา (right subtree) ของ x จะมีค่ามากกว่า x

ตัวอย่าง:

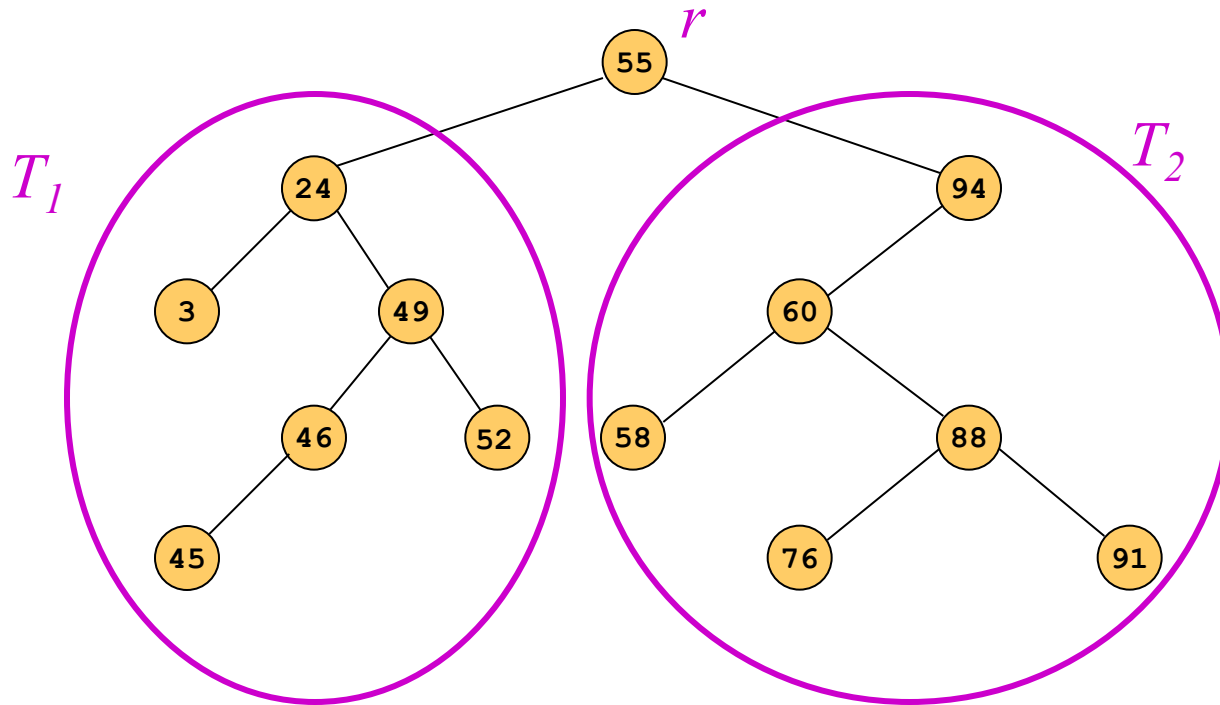


Tree Traversal

- Is a procedure that systematically visits every vertex of an ordered rooted tree.
 - Visiting a vertex = processing the data at the vertex.
- อัลกอริทึมในการท่องไปบนต้นไม้(Traversal algorithms)
 - Pre-order traversal **NLR** (node,left,right)
 - In-order traversal **LNR** (left,node,right)
 - Post-order traversal **LRN** (left,right, node)

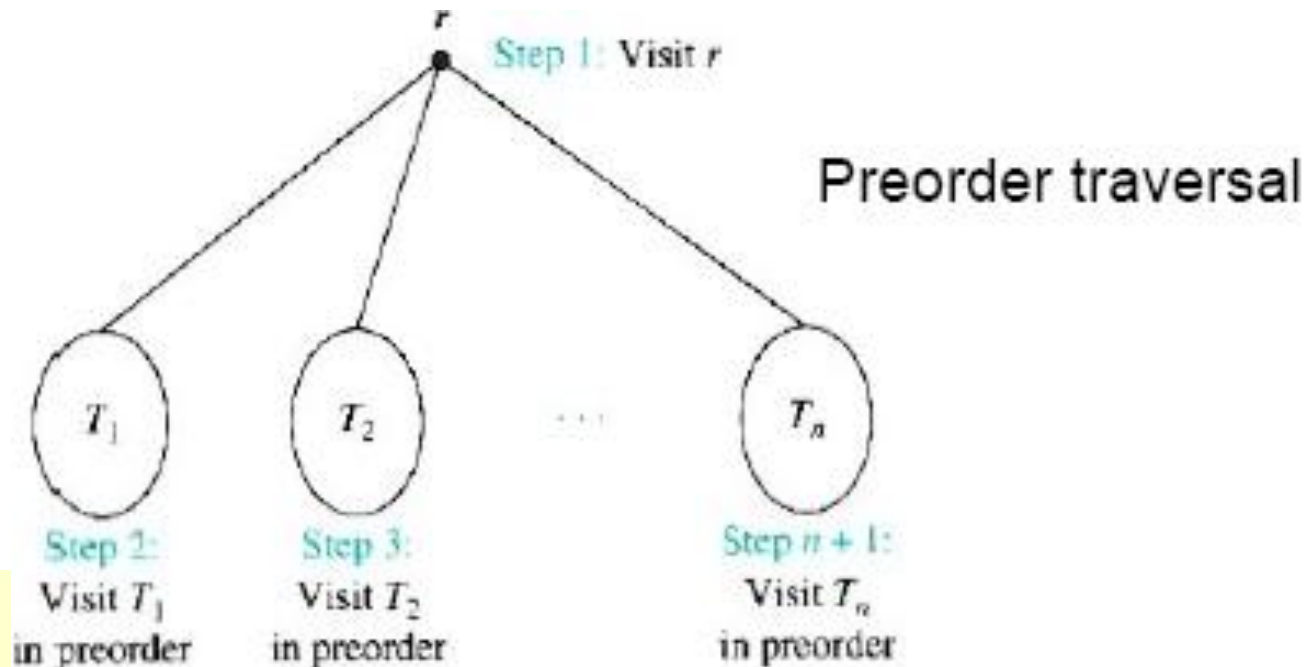
Tree Traversal

- Let T be an ordered rooted tree with root r . Suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right in T .



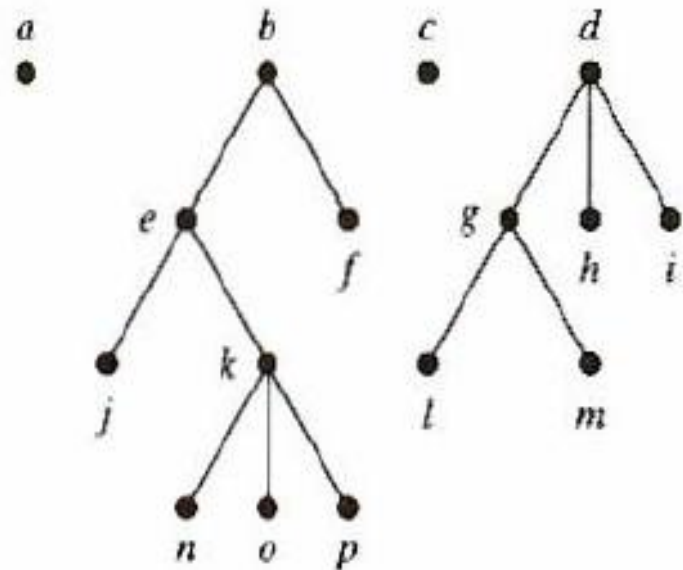
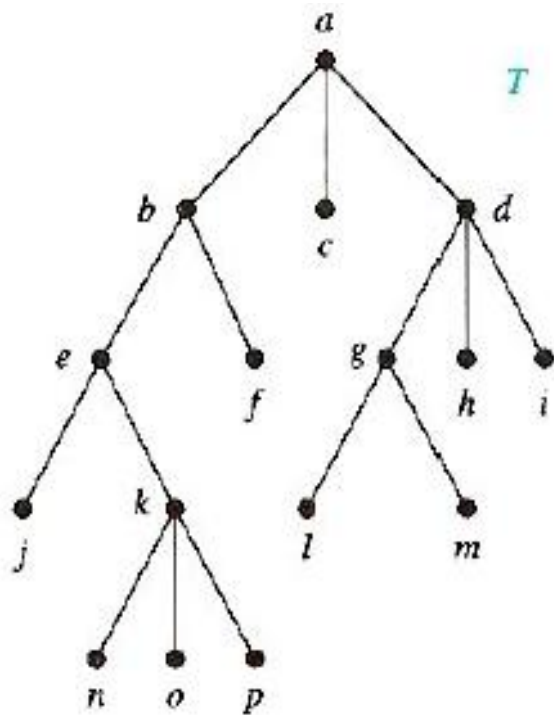
Preorder Traversal

- Begins by visiting r .
- Next traverses T_1 in preorder, then T_2 in preorder, ..., then T_n in preorder.
- Ends after T_n has been traversed.

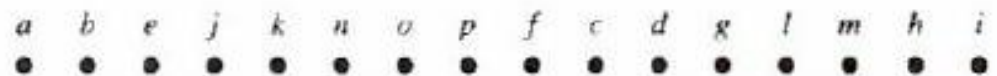
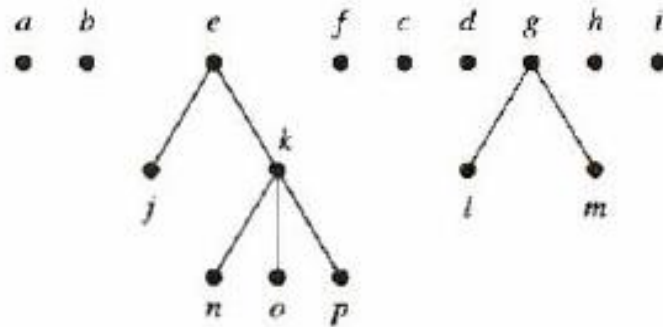
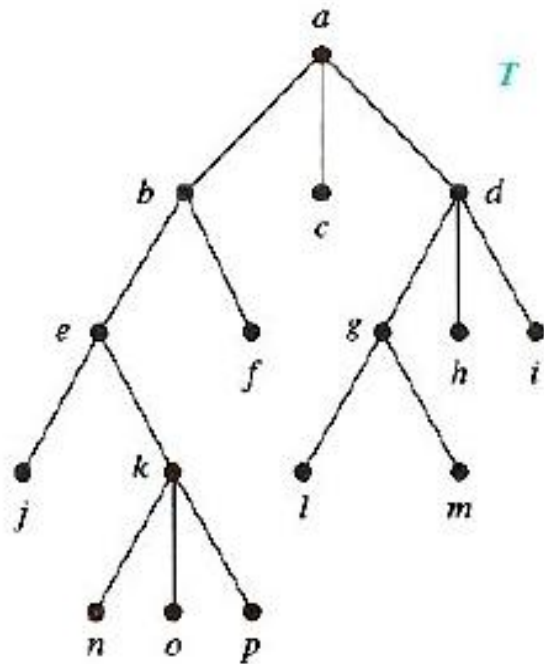


Preorder Traversal

- Visit root, visit subtrees left to right.

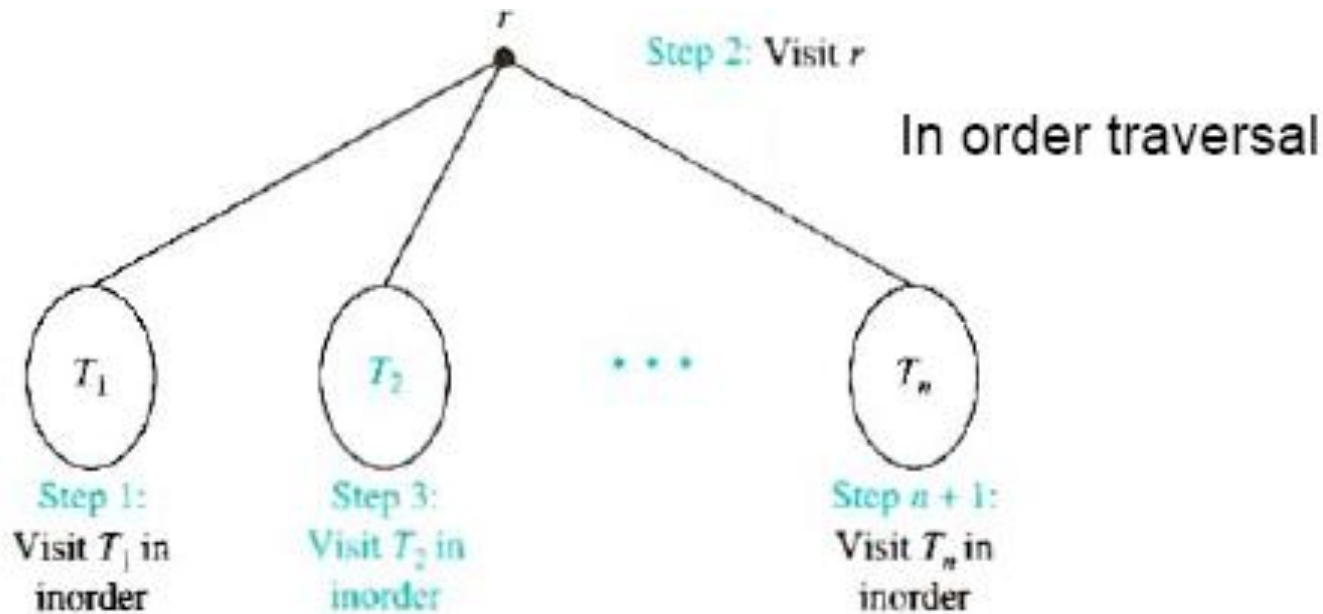


Preorder Traversal



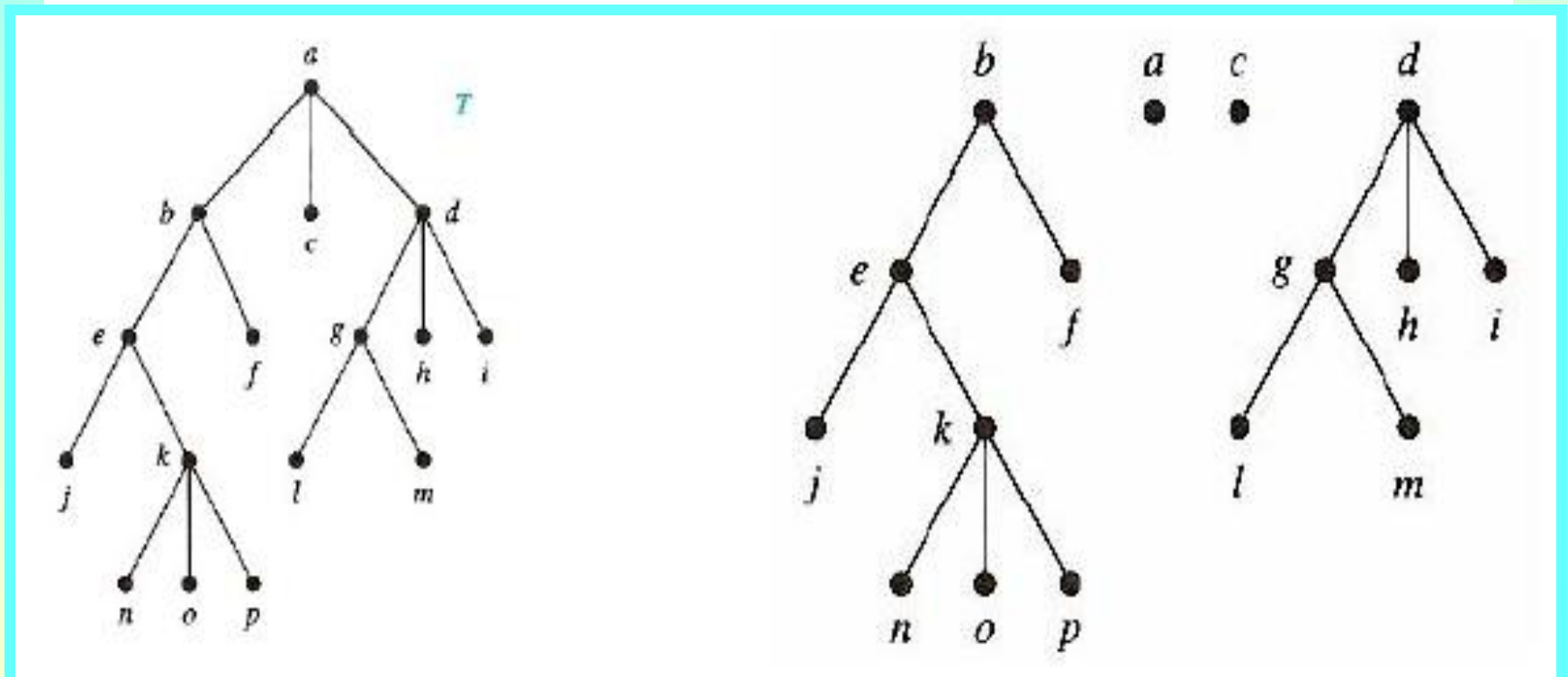
Inorder Traversal

- Begins by traversing T_1 in inorder
- Next visit r , then traverses T_2 in inorder, ..., then T_n in inorder.
- Ends after T_n has been traversed.

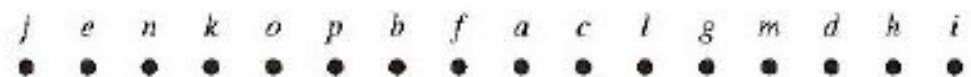
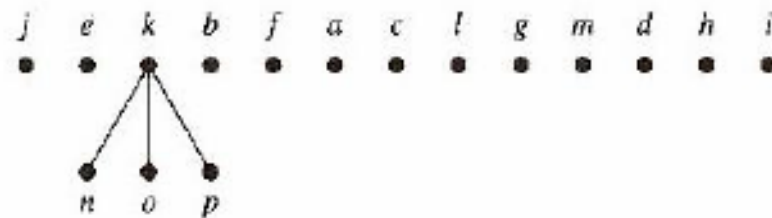
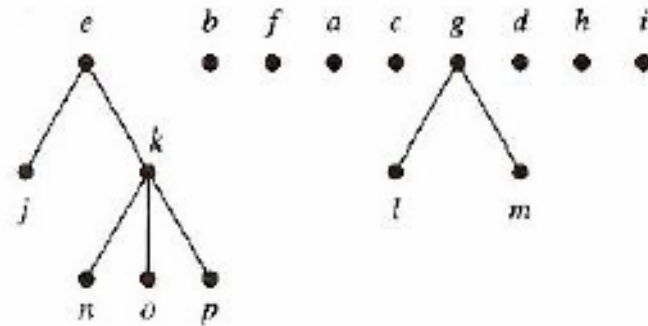
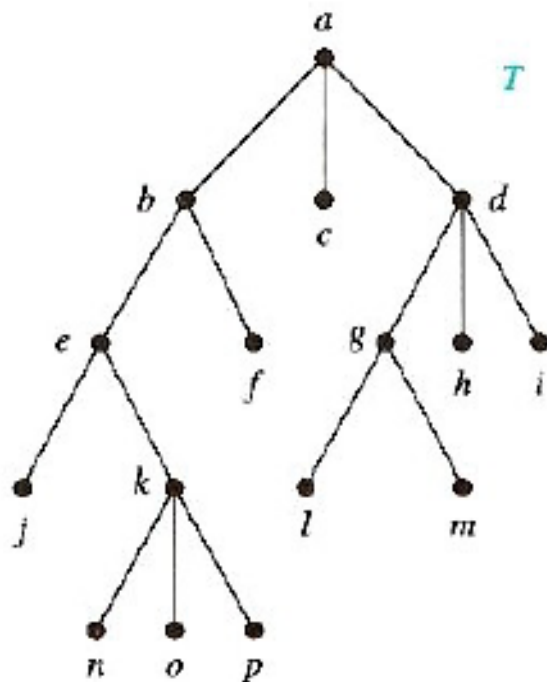


Inorder Traversal

- Visit leftmost subtree, visit root, visit other subtrees left to right.

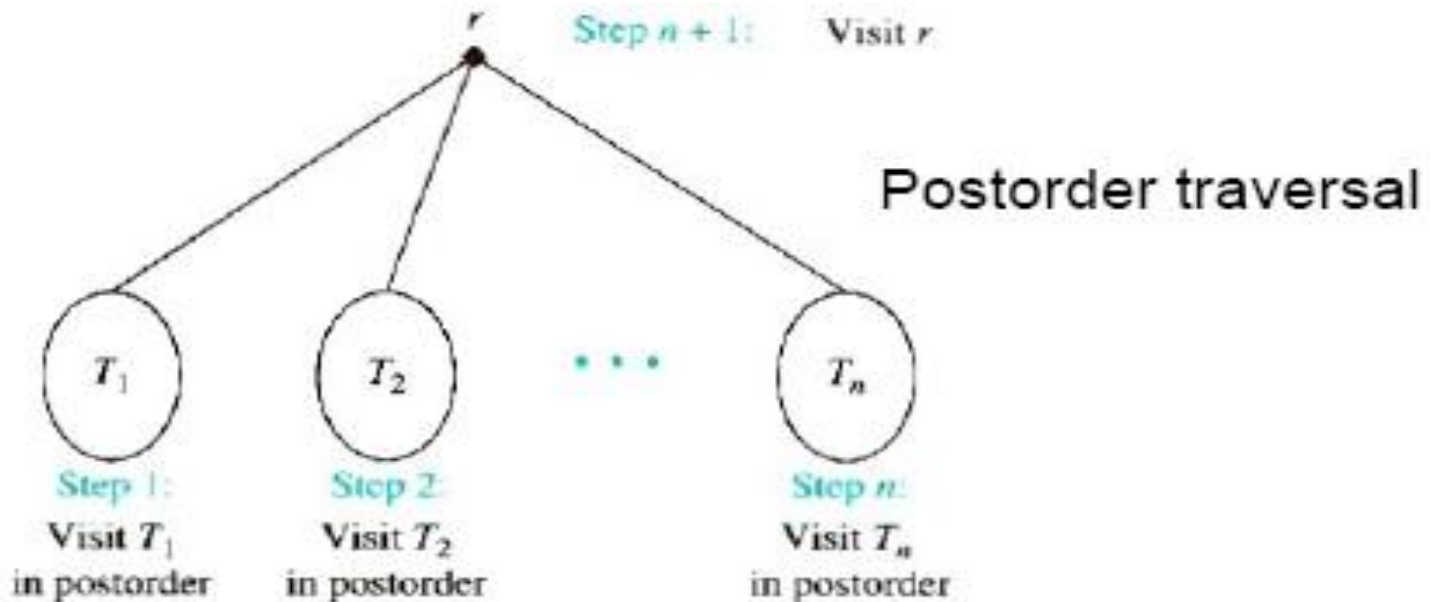


Inorder Traversal



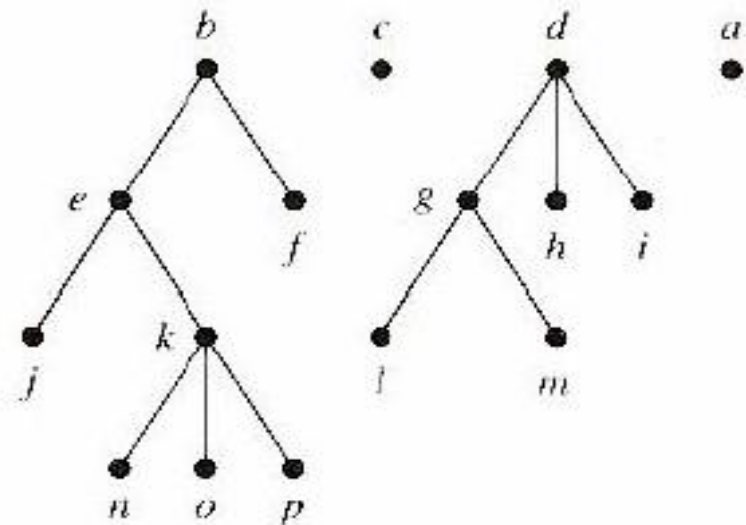
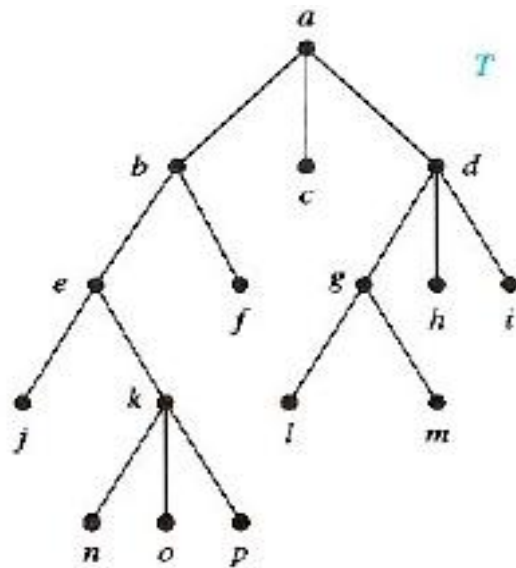
Postorder Traversal

- Begins by traversing T_1 in postorder
- Next traverses T_2 in postorder, then T_3 in postorder, ..., then T_n in postorder
- End by visiting r

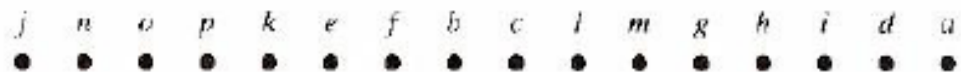
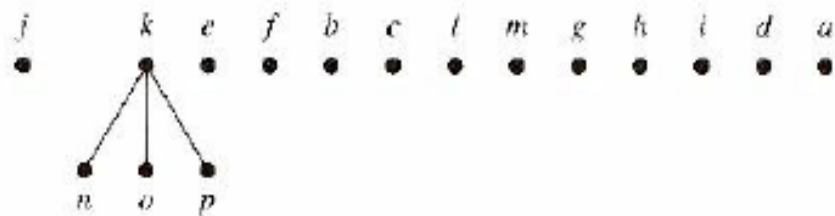
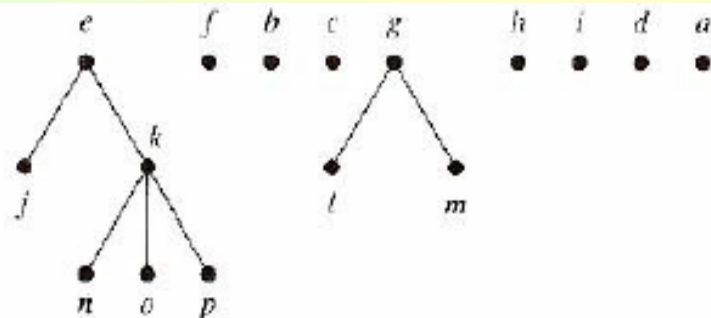
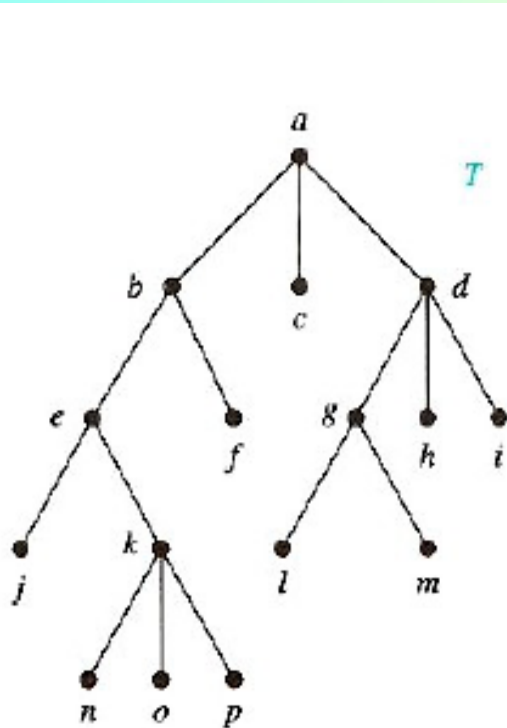


Postorder Traversal

- Visit subtree left to right, visit root.

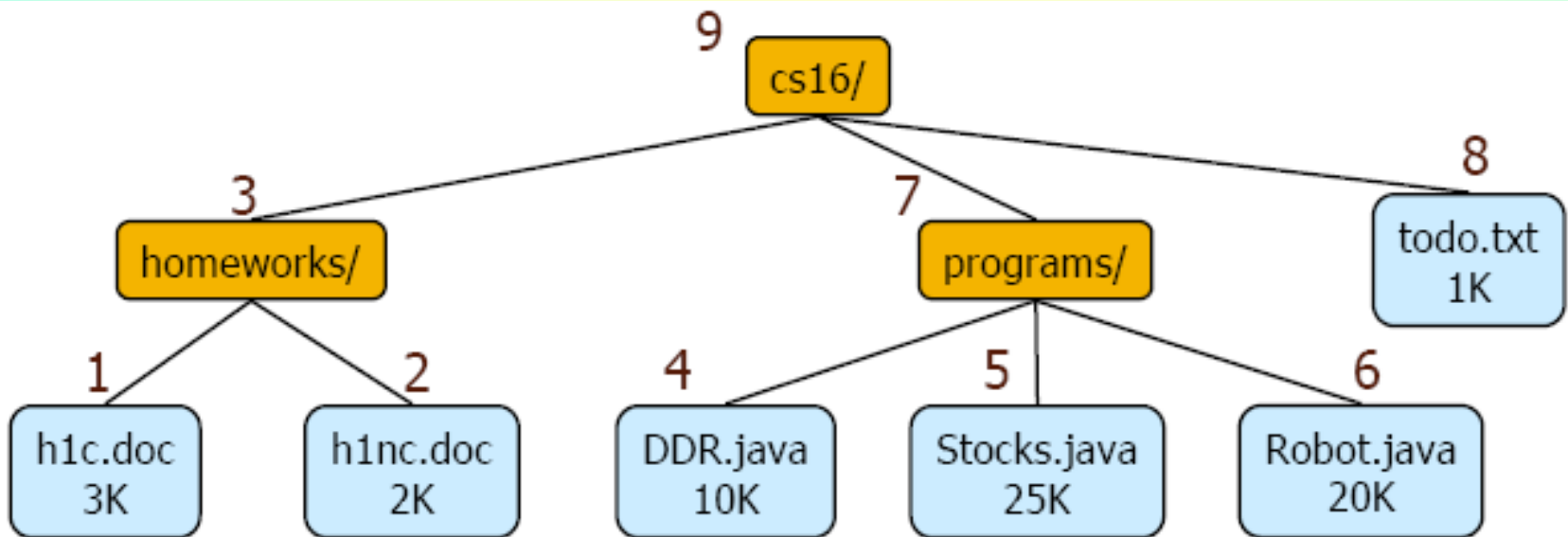


Postorder Traversal



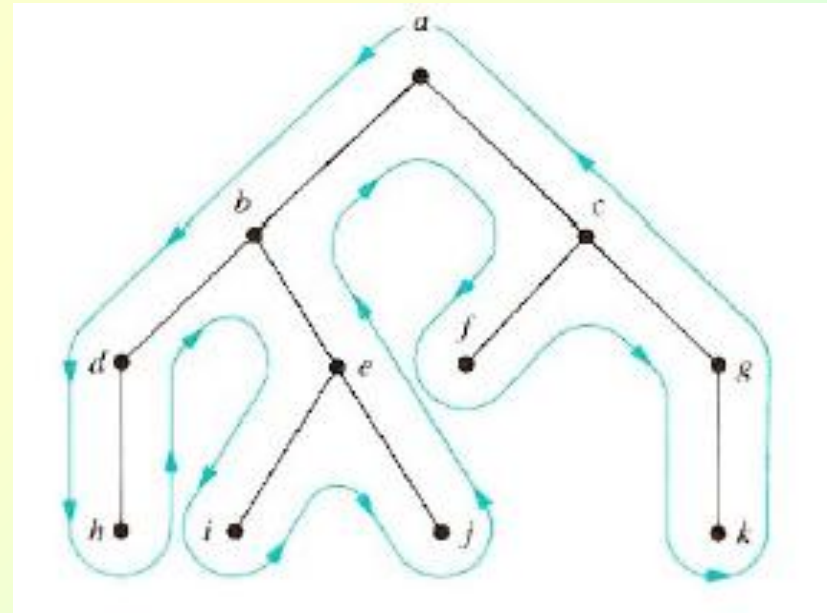
Postorder Traversal

- Applications: compute space used by files in a directory and its subdirectories.



Tree Traversal Shortcut

- Preorder list is obtained by
 - listing each vertex the first time this curve passes it.
 - Result: a, b, d, h, e, i, j, c, f, g, k.
- Inorderlist is obtained by
 - listing a leaf the first time the curve passes it and
 - listing each internal vertex the second time the curve passes it.
 - Result: h, d, b, i, e, j, a, f, c, k, g.
- Postorderlist is obtained by
 - listing a vertex the last time it is passes on the way back up to its parent.
 - Result: h, d, i, j, e, b, f, k, g, c, a.

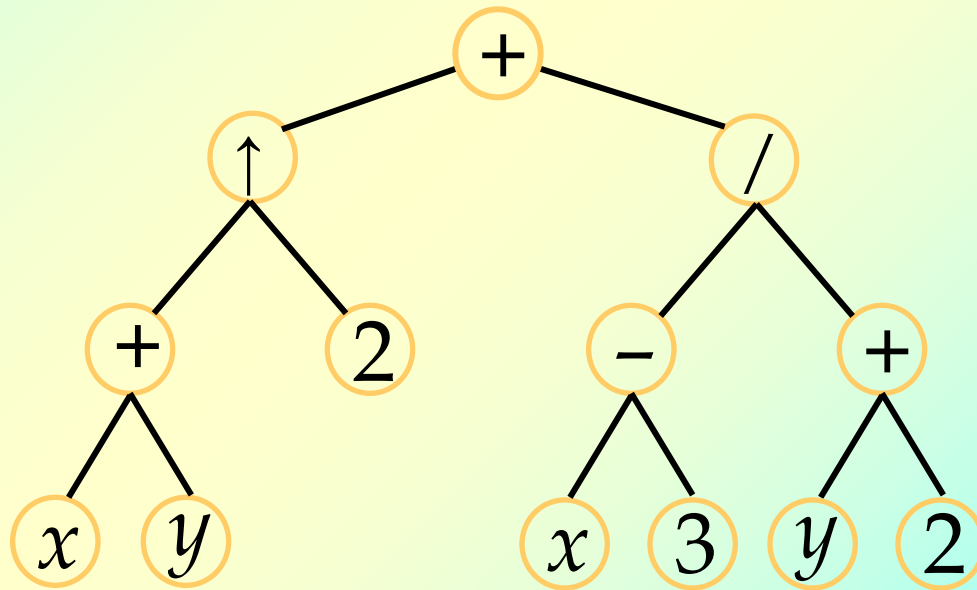


Representing Arithmetic Expressions

- Complicated arithmetic expressions can be represented by an ordered rooted tree
 - Internal vertices represent operators
 - Leaves represent operands
- Build the tree bottom-up
 - Construct smaller subtrees
 - Incorporate the smaller subtrees as part of larger subtrees

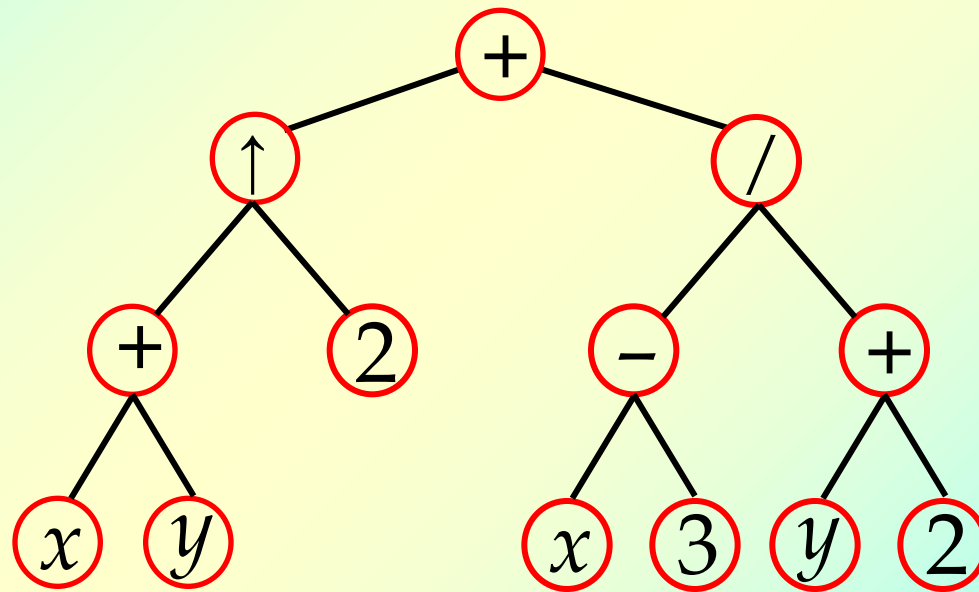
Example

$$(x+y)^2 + (x-3)/(y+2)$$



Infix Notation

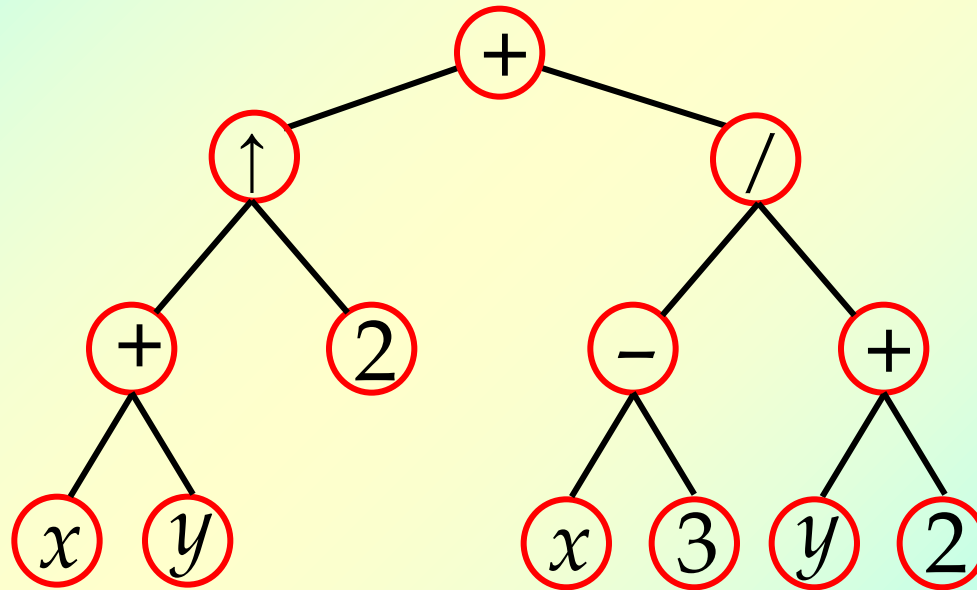
- Traverse in inorder adding parentheses for each operation



$((((x + y) \uparrow 2) + ((x - 3) / (y + 2))))$

Prefix Notation (Polish Notation)

- Traverse in preorder



+ ↑ + x y 2 / - x 3 + y 2

Evaluating Prefix Notation

- In an prefix expression, a binary operator precedes its two operands
- The expression is evaluated right-left
- Look for the first operator from the right
- Evaluate the operator with the two operands immediately to its right

Example

$$+ \ / \ + \ 2 \ 2 \ 2 \ / \ - \ 3 \ 2 \ (+ \ 1 \ 0)$$

$$+ \ / \ + \ 2 \ 2 \ 2 \ / \ (- \ 3 \ 2) \ 1$$

$$+ \ / \ + \ 2 \ 2 \ 2 \ (/ \ 1 \ 1)$$

$$+ \ / \ (+ \ 2 \ 2) \ 2 \ 1$$

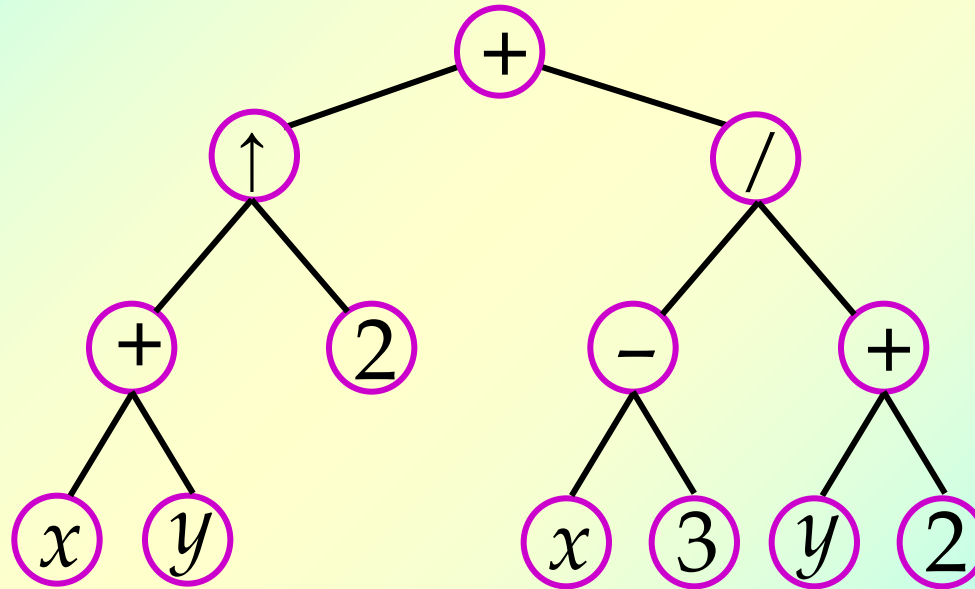
$$+ \ (/ \ 4 \ 2) \ 1$$

$$(+ \ 2 \ 1)$$

3

Postfix Notation (Reverse Polish)

- Traverse in postorder



$x \ y \ + \ 2 \ \uparrow \ x \ 3 \ - \ y \ 2 \ + \ / \ +$

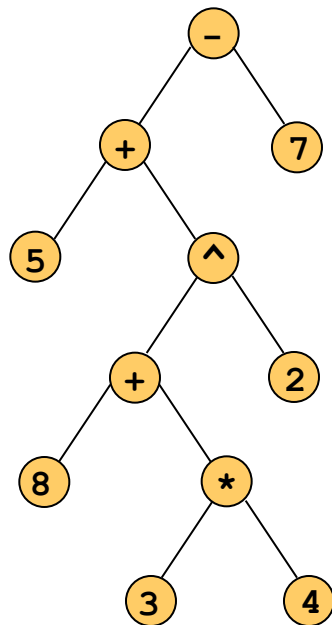
Evaluating Postfix Notation

- In an postfix expression, a binary operator follows its two operands
- The expression is evaluated left-right
- Look for the first operator from the left
- Evaluate the operator with the two operands immediately to its left

Example

$$\begin{array}{cccccccccccc} (2 & 2 & +) & 2 & / & 3 & 2 & - & 1 & 0 & + & / & + \\ & (4 & 2 & /) & 3 & 2 & - & 1 & 0 & + & / & + \\ & & 2 & (3 & 2 & -) & 1 & 0 & + & / & + \\ & & & 2 & 1 & (1 & 0 & +) & / & + \\ & & & & 2 & (1 & 1 & /) & + \\ & & & & & & (2 & 1 & +) \\ & & & & & & & 3 \end{array}$$

Tree Traversal



prefix expression

`(- (+ 5 (^ (+ 8 (* 3 4)) 2)) 7)`

infix expression

`((5 + ((8 + (3 * 4)) ^ 2)) - 7)`

postfix expression

`5 8 3 4 * + 2 ^ + 7 -`